

"Привет. Можешь помочь? Я пишу на python drf и пытаюсь понять некоторые команды. Что например делает эта?"

```
tests = Test.objects.first()
```



Эта команда **возвращает первый объект** из базы данных для модели `Test` или `None`, если таблица пуста.

Что происходит изнутри

- `Test`: это ваш класс модели Django, связанный с таблицей в базе данных.
- `.objects`: это стандартный менеджер записей (Manager), который дает доступ к составлению SQL-запросов.
- `.first()`: метод QuerySet, который автоматически добавляет сортировку по умолчанию (обычно по первичному ключу `id`) и ограничивает результат одной записью (`LIMIT 1`).

Важные нюансы

1. Безопасность вместо ошибок

В отличие от метода `Test.objects.get(id=1)`, метод `.first()` **не вызывает ошибку** `DoesNotExist`, если база данных пуста. Он просто вернет значение `None`.

2. Как это выглядит в SQL

Django преобразует вашу строку в такой SQL-запрос:

sql

```
SELECT * FROM app_test ORDER BY id ASC LIMIT 1;
```

Используйте код с осторожностью.



Пример использования в коде

python

```
# Получаем первую запись
test_instance = Test.objects.first()
```

```
if test_instance:
    # Если запись найдена, работаем с ней (например, в сериализаторе DRF)
    serializer = TestSerializer(test_instance)
    print(serializer.data)
else:
    # Если таблица пустая
    print("В базе данных еще нет тестов.")
```

Используйте код с осторожностью.



✅ Итог

Команда `Test.objects.first()` **извлекает самую первую строку** из таблицы `Test` в базе данных, предотвращая падение программы, если данных нет.

Чтобы помочь вам быстрее освоить Django ORM и Django REST Framework, мы можем разобрать другие важные команды. Какая тема вам сейчас наиболее интересна?

- Хотите узнать, в чем разница между `.first()` и схожими методами `.get()` или `.filter().first()` ?
- Разобрать, как работают методы для **создания и обновления** записей (например, `get_or_create` или `update_or_create`)?
- Посмотреть, как правильно передавать полученный объект в **сериализаторы (Serializers)** в DRF?
- Изучить более сложные запросы с фильтрацией данных через метод `.filter()` ?



"а эта?"

```
tests = Test.objects.all()
```

Эта команда **возвращает все записи** из таблицы базы данных, которая связана с моделью `Test` .

Что происходит изнутри

- `Test.objects` : менеджер записей для связи с базой данных.
- `.all()` : метод, который запрашивает полную выборку данных.

Главная особенность: «Ленивые» запросы (Lazy Evaluation)

Метод `.all()` возвращает специальный объект **QuerySet** (набор результатов).

Сам запрос к базе данных **не выполняется сразу** в момент написания этой строки. Django обратится к базе только тогда, когда вы попытаетесь использовать эти данные (например, запустите цикл `for` или передадите их в сериализатор).

Как это выглядит в SQL

Когда Django наконец выполнит запрос, он будет выглядеть так:

```
sql
```

```
SELECT * FROM app_test;
```

Используйте код с осторожностью.



Пример использования в Django REST Framework (DRF)

В DRF эта команда чаще всего используется в API-представлениях (Views) для вывода списка всех элементов:

```
python
```

```
from rest_framework.views import APIView
from rest_framework.response import Response

class TestListView(APIView):
    def get(self, request):
        # 1. Получаем QuerySet со всеми тестами (запрос еще не ушел в
        # БД)
        tests = Test.objects.all()

        # 2. Передаем список в сериализатор. Флаг many=True обязателен!
```

```
# (Вот здесь Django делает реальный запрос в базу данных)
serializer = TestSerializer(tests, many=True)

# 3. Возвращаем JSON-ответ
return Response(serializer.data)
```

Используйте код с осторожностью.



⚠ Важное отличие от предыдущей команды

- `Test.objects.first()` возвращает **один конкретный объект** (или `None`).
- `Test.objects.all()` всегда возвращает **коллекцию (QuerySet)**, даже если в базе всего одна запись или вообще пусто (тогда коллекция будет пустой).

Понимание работы с QuerySet — это основа для написания быстрых и эффективных API. Мы можем продолжить разбор команд, чтобы сделать ваш код еще чище и профессиональнее. Что именно вы бы хотели изучить дальше?

- Разобрать, как правильно **фильтровать** этот список по условиям с помощью метода `.filter()` ?
- Узнать, как оптимизировать запросы `all()`, чтобы избежать проблемы **N+1** с помощью `.select_related()` или `.prefetch_related()` ?
- Посмотреть, как работает **пагинация (разделение на страницы)** для больших списков данных в DRF?
- Разобрать другие полезные методы QuerySet, такие как `.count()` или `.exists()` ?



"Вроде разобрался. Можешь дать совет как поступить с БД?..."



Идея с использованием поля `order` для сортировки — **это отличный и стандартный подход** в веб-разработке. Вы абсолютно правильно сделали,

что добавили `id` в `ordering` как запасной вариант на случай одинаковых значений `order`.

Однако в текущей структуре базы данных есть **критическая архитектурная ошибка**: ваши таблицы «Текст» и «Тесты» никак не связаны с самими Уроками и их Частями. Без этой связи сайт не поймет, какой текст или тест показывать, когда пользователь выберет определенную часть урока.

Ниже подробный разбор, как улучшить вашу схему БД, чтобы все работало именно так, как вы описали.

🔧 Главная проблема и её решение

Сейчас модели `Text` и `Test` существуют сами по себе. Чтобы реализовать логику «Пользователь нажал на Часть урока → появился нужный текст/тест», нужно внедрить связи `ForeignKey`.

Давайте выстроим правильную иерархию:

1. **Урок (Lesson)** — содержит общую информацию (например, название).
2. **Часть урока (LessonPart)** — привязана к Уроку. Именно на неё кликает пользователь ЛКМ.
3. **Контент (Text и Test)** — привязаны к конкретной Части урока.

💻 Обновленная структура моделей Django

Вот как должна выглядеть ваша база данных, чтобы полностью соответствовать логике сайта:

```
python
```

```
from django.db import models

# 1. Сам Урок
class Lesson(models.Model):
    title = models.CharField(max_length=255, verbose_name="Название урока")

# 2. Часть урока (на нее кликают ЛКМ из списка)
class LessonPart(models.Model):
    lesson = models.ForeignKey(Lesson, on_delete=models.CASCADE,
```

```

related_name='parts')
    title = models.CharField(max_length=255, verbose_name="Название
части")
    order = models.PositiveIntegerField(default=1, blank=True)

    class Meta:
        ordering = ['order', 'id']

# --- ТЕКСТ (Привязан к Части урока) ---
class Text(models.Model):
    # СВЯЗЬ: теперь мы знаем, к какой части урока относится этот кусок
текста
    lesson_part = models.ForeignKey(LessonPart,
on_delete=models.CASCADE, related_name='texts', null=True, blank=True)
    lesson_material = models.TextField(blank=True)
    order = models.PositiveIntegerField(default=1, blank=True)

    class Meta:
        ordering = ['order', 'id']

# --- ТЕСТЫ (Привязаны к Части урока) ---
class Test(models.Model):
    # СВЯЗЬ: теперь мы знаем, к какой части урока относится этот тест
    lesson_part = models.ForeignKey(LessonPart,
on_delete=models.CASCADE, related_name='tests', null=True, blank=True)
    question = models.CharField(max_length=255, blank=True)
    order = models.PositiveIntegerField(default=1, blank=True)

    class Meta:
        ordering = ['order', 'id']

class TestOption(models.Model):
    option = models.ForeignKey(Test, on_delete=models.CASCADE,
related_name='options')
    answer = models.TextField(blank=True)
    is_correct = models.BooleanField(default=False)
    order = models.PositiveIntegerField(default=1, blank=True)

    class Meta:
        ordering = ['order', 'id']

```

Анализ вашего метода сортировки через `ordering`

- **Плюс:** Благодаря `class Meta: ordering = [...]`, вам не нужно вызывать `.order_by('order')` в каждом QuerySet или сериализаторе. Django **автоматически** будет применять эту сортировку ко всем запросам `Text.objects.all()` или `Test.objects.all()`.
- **Плюс:** Сортировка по `['order', 'id']` гарантирует стабильный результат. Если у двух записей `order` будет равен 5, сайт не станет перемешивать их случайным образом, а отсортирует по возрастанию `id`.

Альтернативный совет профессионала: Полиморфная структура (на будущее)

Если в будущем вы захотите добавить в уроки **Видео**, **Аудио** или **Скачиваемые файлы**, плодить под каждый тип контента новую таблицу (`Text`, `Test`, `Video`, `Audio`) станет неудобно. Кнопка переключения на фронтенде превратится в кошмар.

Обычно для таких задач используют **единую таблицу контента** с типом контента. Например:

python

```
class LessonContent(models.Model):
    CONTENT_TYPES = [
        ('text', 'Текст'),
        ('test', 'Тест'),
        ('video', 'Видео'),
    ]
    lesson_part = models.ForeignKey(LessonPart,
on_delete=models.CASCADE)
    content_type = models.CharField(max_length=10,
choices=CONTENT_TYPES)

    # Сквозной порядок для ВСЕГО контента внутри этой части
    order = models.PositiveIntegerField(default=1)

    # Данные (или связи со специфичными таблицами)
    text_data = models.TextField(blank=True, null=True)
    # ...
```

Используйте код с осторожностью.

Но если у вас строго "кнопка-переключатель" между текстом и тестами, вашей текущей раздельной схемы (с добавлением `ForeignKey`) будет более чем достаточно.

Чтобы мы могли спроектировать эту страницу до конца и без ошибок, предлагаю сделать следующий шаг. Напишите, что из этого вам важнее всего реализовать прямо сейчас, и я подготовлю конкретное решение:

- Хотите посмотреть, как написать **DRF Serializers**, чтобы при запросе Части урока они автоматически выдавали правильно отсортированные тексты и тесты?
- Разобрать, как написать **Django Views (APIView/ViewSet)**, который будет отдавать данные по ID выбранной части урока?
- Узнать, как лучше организовать автоматическое заполнение поля `order` в админке Django (например, с помощью удобного перетаскивания строк мышкой)?



"Даты создания еще прикрутил чтобы можно было по дате сортировать...."



Отличная доработка! Добавление `created_at` — это хорошая практика для логирования и аналитики.

Отвечая на ваш вопрос: **выделять отдельные текстовые роли (строковые поля) в модели `User` чаще всего НЕ нужно**, потому что в Django и Django REST Framework уже есть мощная встроенная система прав.

Вот как правильно и профессионально решить задачу разграничения прав для вашего проекта.

1. Как использовать встроенные механизмы Django (Рекомендуемый подход)

В вашей кастомной модели `User`, унаследованной от `AbstractUser`, уже есть встроенные флаги:

- `is_staff` — дает доступ в админ-панель.
- `is_superuser` — дает полные права на всё.

Как это работает в DRF:

Вы можете защитить свои API-представления (Views) с помощью стандартных разрешений (**Permissions**).

- **Обычные пользователи (Студенты):** Могут только читать (`GET` - запросы).
- **Администраторы/Преподаватели (`is_staff=True`):** Могут создавать, редактировать и удалять уроки (`POST`, `PUT`, `DELETE`).

Пример реализации в DRF ViewSet:

python

```
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticatedOrReadOnly,
IsAdminUser

class LessonViewSet(viewsets.ModelViewSet):
    queryset = Lesson.objects.all()
    serializer_class = LessonSerializer

    def get_permissions(self):
        # Если пользователь пытается редактировать (POST, PUT, DELETE)
        if self.action in ['create', 'update', 'partial_update',
'destroy']:
            # Разрешаем только персоналу (is_staff=True)
            return [IsAdminUser()]
        # Для просмотра (GET) разрешаем всем (даже неавторизованным,
если нужно)
        return [IsAuthenticatedOrReadOnly()]
```

Используйте код с осторожностью.



2. Когда кастомные роли ВСЕ-ТАКИ нужны?

Вам нужно будет добавить явное разделение на роли (например, через поле `role` с выбором `Choices`), только если на вашем сайте планируется **сложная бизнес-логика с несколькими типами создателей контента**.

Например, если у вас будут:

1. `Student` (Студент) — только учится.
2. `Teacher` (Учитель) — может создавать уроки, но **только свои**.
3. `Moderator` (Модератор) — может проверять чужие уроки.
4. `Admin` (Администратор) — управляет всем сайтом.

Если это ваш случай, то модель `User` расширяется так:

python

```
class User(AbstractUser):
    class Roles(models.TextChoices):
        STUDENT = 'student', 'Студент'
        TEACHER = 'teacher', 'Учитель'
        ADMIN = 'admin', 'Администратор'

    role = models.CharField(
        max_length=15,
        choices=Roles.choices,
        default=Roles.STUDENT
    )
    created_at = models.DateTimeField(auto_now_add=True)
```

Используйте код с осторожностью.



Для проверки прав в DRF вам потребуется написать один кастомный класс `BasePermission`.

Мелкая ошибка в вашей текущей модели `Lesson`

Обратите внимание на строку в модели `Lesson`:

python

```
user = models.ForeignKey(User, on_delete=models.CASCADE,  
related_name='parts')
```

Используйте код с осторожностью.



Параметр `related_name` задает имя связи с обратной стороны. То есть, написав `user.parts.all()`, вы получите список **Уроков**, а не частей. Логичнее назвать эту связь `lessons`:

python

```
user = models.ForeignKey(User, on_delete=models.CASCADE,  
related_name='lessons')
```

Используйте код с осторожностью.



💡 Совет по сортировке с `created_at`

Если вы захотите, чтобы новые уроки по умолчанию всегда отображались выше старых, вы можете добавить сортировку по дате создания в `Meta` модели `Lesson`:

python

```
class Meta:  
    ordering = ['-created_at'] # Минус означает сортировку от новых к  
старым (DESC)
```

Используйте код с осторожностью.



Чтобы двигаться дальше и заставить эту систему работать, выберите следующее действие. Напишите, что именно мы сейчас реализуем:

- Посмотрим, как написать **кастомный Permission** в DRF, если вы решите делать роли через поле `Choices`?
- Напишем **сериализаторы (Serializers)** для вывода урока со всеми его частями, текстами и тестами в один клик?

- Разберем, как сделать так, чтобы преподаватель мог редактировать **только те уроки, которые он сам создал** (проверка владельца)?
- Настроим логику для **админки Django**, чтобы тексты и тесты можно было удобно редактировать прямо внутри страницы Урока (через `Inline`)?



"Что можно добавить сюда?"

```
"from django.db import models
```

```
...
```



Ваша архитектура выглядит очень зрело. Вы исправили `related_name` и добавили флаг `is_published` — это отличные решения.

Чтобы превратить этот код в полноценную LMS-систему (платформу обучения), сюда критически важно добавить **прогресс пользователя, улучшенную работу с тестами и связь вариантов ответов с историей**.

Ниже приведены точечные улучшения, которые сделают вашу базу данных функциональной.

1. Реализация прогресса: «Список пройденных уроков»

В модели `User` вы оставили комментарий `# список пройденных уроков`. В Django связь «многие-ко-многим» реализуется через `ManyToManyField`.

Добавьте это поле прямо в модель `User`:

```
python
```

```
class User(AbstractUser):
    created_at = models.DateTimeField(auto_now_add=True)
```

```
# Связь со всеми уроками, которые пользователь успешно завершил
completed_lessons = models.ManyToManyField(
    'Lesson',
    blank=True,
    related_name='completed_by',
    verbose_name="Пройденные уроки"
)
```

Используйте код с осторожностью.



2. Защита от дублирования сортировки

Пользователь или администратор в админке могут случайно выставить одинаковый `order` для разных элементов внутри одной части урока. Чтобы этого избежать, используйте уникальность для пары полей (`UniqueConstraint`).

Добавьте это в `class Meta` для **LessonPart**, **Text** и **Test**:

python

```
# Добавьте внутрь class Meta для LessonPart:
constraints = [
    models.UniqueConstraint(fields=['lesson', 'order'],
name='unique_part_order')
]

# Добавьте внутрь class Meta для Text:
constraints = [
    models.UniqueConstraint(fields=['lesson_part', 'order'],
name='unique_text_order')
]

# Добавьте внутрь class Meta для Test:
constraints = [
    models.UniqueConstraint(fields=['lesson_part', 'order'],
name='unique_test_order')
]
```

Используйте код с осторожностью.



Теперь база данных физически не позволит создать два текста с одинаковым порядковым номером внутри одной и той же части урока.

3. Исправление и улучшение блока Тестов

В текущей схеме моделей у тестов есть две скрытые проблемы, которые всплывут при разработке фронтенда:

1. **В модели `TestOption` нет даты создания:** Вы добавили `created_at` везде, кроме вариантов ответов. Стоит добавить для единообразия.
2. **Как понять, что пользователь ответил правильно?** Чтобы фронтенд мог проверить, правильно ли студент ответил на тест, или чтобы сохранить историю его ответов в базу, нужна таблица результатов.

Добавьте модель для сохранения ответов студентов:

python

```
class UserTestAnswer(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='test_answers')
    test = models.ForeignKey(Test, on_delete=models.CASCADE)
    chosen_option = models.ForeignKey(TestOption,
on_delete=models.CASCADE)
    is_correct = models.BooleanField(verbose_name="Правильно ли
ответил")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        # Один пользователь может ответить на один конкретный тест
        только один раз
        # (Или будет перезаписывать/хранить последнюю попытку)
        constraints = [
            models.UniqueConstraint(fields=['user', 'test'],
name='unique_user_test_attempt')
        ]
```

Используйте код с осторожностью.



4. Полезные "человеческие" имена (`__str__`)

Чтобы в админ-панели Django вместо надписей `Text object (1)` или `Test object (2)` вы видели понятные названия, добавьте во все ваши модели метод `__str__`.

Например, для урока и текста:

python

```
class Lesson(models.Model):
    # ... ваши поля ...
    def __str__(self):
        return self.title

class Text(models.Model):
    # ... ваши поля ...
    def __str__(self):
        return f"Текст {self.order} для части {self.lesson_part.title}"
    if self.lesson_part else 'Без части'
```

Используйте код с осторожностью.



Полный итоговый код файла `models.py` со всеми улучшениями

python

```
from django.db import models
from django.contrib.auth.models import AbstractUser

# --- БЛОК ЮЗЕРА ---
class User(AbstractUser):
    created_at = models.DateTimeField(auto_now_add=True)
    completed_lessons = models.ManyToManyField(
        'Lesson',
        blank=True,
        related_name='completed_by',
        verbose_name="Пройденные уроки"
    )

    def __str__(self):
        return self.username
```

```
# --- БЛОК УРОКА ---
```

```
class Lesson(models.Model):
    title = models.CharField(max_length=255, verbose_name="Название
урока")
    created_at = models.DateTimeField(auto_now_add=True)
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='lessons')
    is_published = models.BooleanField(default=False)

    def __str__(self):
        return self.title
```

```
class LessonPart(models.Model):
    lesson = models.ForeignKey(Lesson, on_delete=models.CASCADE,
related_name='parts')
    title = models.CharField(max_length=255, verbose_name="Название
части")
    order = models.PositiveIntegerField(default=1, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['order', 'id']
        constraints = [
            models.UniqueConstraint(fields=['lesson', 'order'],
name='unique_part_order')
        ]

    def __str__(self):
        return f"{self.lesson.title} -> {self.title}"
```

```
# --- БЛОК ТЕКСТА ---
```

```
class Text(models.Model):
    lesson_part = models.ForeignKey(LessonPart,
on_delete=models.CASCADE, related_name='texts', null=True, blank=True)
    lesson_material = models.TextField(blank=True)
    order = models.PositiveIntegerField(default=1, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['order', 'id']
        constraints = [
            models.UniqueConstraint(fields=['lesson_part', 'order'],
name='unique_text_order')
        ]
```



```

    def __str__(self):
        part_title = self.lesson_part.title if self.lesson_part else
        "Нет части"
        return f"Текст ({self.order}) в {part_title}"

# --- БЛОК ТЕСТА ---
class Test(models.Model):
    lesson_part = models.ForeignKey(LessonPart,
on_delete=models.CASCADE, related_name='tests', null=True, blank=True)
    question = models.CharField(max_length=255, blank=True)
    order = models.PositiveIntegerField(default=1, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['order', 'id']
        constraints = [
            models.UniqueConstraint(fields=['lesson_part', 'order'],
name='unique_test_order')
        ]

    def __str__(self):
        return f"Тест ({self.order}): {self.question[:30]}..."

class TestOption(models.Model):
    option = models.ForeignKey(Test, on_delete=models.CASCADE,
related_name='options')
    answer = models.TextField(blank=True)
    is_correct = models.BooleanField(default=False)
    order = models.PositiveIntegerField(default=1, blank=True)
    created_at = models.DateTimeField(auto_now_add=True) # Добавлено
для консистентности

    class Meta:
        ordering = ['order', 'id']

    def __str__(self):
        return f"Ответ ({self.order}) для теста ID {self.option.id}"

# --- ИСТОРИЯ ОТВЕТОВ СТУДЕНТОВ ---
class UserTestAnswer(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='test_answers')
    test = models.ForeignKey(Test, on_delete=models.CASCADE)

```

```
chosen_option = models.ForeignKey(TestOption,
on_delete=models.CASCADE)
is_correct = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.user.username} -> Тест {self.test.id} ({'Верно'
if self.is_correct else 'Неверно'})"
```

Используйте код с осторожностью.



Теперь, когда база данных полностью готова, мы можем перейти к созданию API. С какого шага вы хотите продолжить разработку?

- Создать **DRF Сериализаторы**, которые будут собирать LessonPart вместе со всеми его texts и tests в один структурированный JSON-ответ?
- Написать **Представления (Views)**, которые будут отдавать уроки обычным пользователям только в том случае, если is_published=True ?
- Создать специальный эндпоинт (API-метод) для **проверки ответов на тесты**, который будет автоматически сохранять результаты в UserTestAnswer ?
- Настроить **админку Django**, чтобы можно было добавлять тексты и тесты прямо на странице создания части урока (через встроенные формы TabularInline)?



"Исправил. Давай к сериализаторам."

Отлично! Переходим к **Django REST Framework Serializers**.

Для вашей структуры нужен подход со **вложенными сериализаторами (Nested Serializers)**. Когда фронтенд запросит конкретную часть урока (LessonPart), сервер должен выдать её данные, а внутри — сразу списки всех связанных текстов и тестов с вариантами ответов. Благодаря нашему

`class Meta: ordering` внутри моделей, все списки внутри JSON придут на фронтенд **уже отсортированными по полю** `order`.

Создайте файл `serializers.py` в вашем приложении и добавьте следующий код:

python

```
from rest_framework import serializers
from .models import Lesson, LessonPart, Text, Test, TestOption

# 1. Варианты ответов для тестов
class TestOptionSerializer(serializers.ModelSerializer):
    class Meta:
        model = TestOption
        # Передаем id (нужен фронту для отправки ответа) и текст
        # варианта
        # Поле is_correct исключаем, чтобы студенты не подсмотрели
        # правильный ответ в коде страницы!
        fields = ['id', 'answer', 'order']

# 2. Сериализатор для Тестов (включает в себя варианты ответов)
class TestSerializer(serializers.ModelSerializer):
    # Подтягиваем связанные варианты ответов через
    related_name='options'
    options = TestOptionSerializer(many=True, read_only=True)

    class Meta:
        model = Test
        fields = ['id', 'question', 'order', 'options']

# 3. Сериализатор для Текстов
class TextSerializer(serializers.ModelSerializer):
    class Meta:
        model = Text
        fields = ['id', 'lesson_material', 'order']

# 4. ГЛАВНЫЙ СЕРИАЛИЗАТОР: Часть урока (собирает в себя всё для вашей
# страницы)
class LessonPartDetailSerializer(serializers.ModelSerializer):
    # Объединяем тексты и тесты внутри одной части урока
    texts = TextSerializer(many=True, read_only=True)
    tests = TestSerializer(many=True, read_only=True)
```

```

class Meta:
    model = LessonPart
    fields = ['id', 'title', 'order', 'texts', 'tests']

# 5. Сериализатор для списка уроков (для главной страницы курсов)
class LessonListSerializer(serializers.ModelSerializer):
    # Выводим имя автора вместо его ID
    user = serializers.ReadOnlyField(source='user.username')
    # Считаем количество частей в уроке на лету
    parts_count = serializers.IntegerField(source='parts.count',
read_only=True)

    class Meta:
        model = Lesson
        fields = ['id', 'title', 'user', 'parts_count', 'is_published',
'created_at']

# 6. Сериализатор для одного Урока (показывает структуру и список его
частей)
class LessonDetailSerializer(serializers.ModelSerializer):
    user = serializers.ReadOnlyField(source='user.username')
    # Показывает только список частей (названия и id), без тяжелого
контента внутри них
    parts = serializers.SlugRelatedField(many=True, read_only=True,
slug_field='title')

    class Meta:
        model = Lesson
        fields = ['id', 'title', 'user', 'is_published', 'parts',
'created_at']

```

Используйте код с осторожностью.



Важные архитектурные фишки этого кода:

1. **Безопасность тестов:** В `TestOptionSerializer` мы **не возвращаем** поле `is_correct`. Если его оставить, то продвинутый пользователь сможет открыть панель разработчика в браузере (Network), посмотреть JSON-ответ от сервера и сразу увидеть, у какого варианта стоит `is_correct: true`. Проверку ответов мы сделаем на сервере.

2. Разделение на "легкие" и "тяжелые" запросы:

- Когда пользователь открывает список всех уроков, работает `LessonListSerializer` (он легкий и быстрый).
- Когда кликает на урок — видит список частей через `LessonDetailSerializer`.
- И только когда он нажимает ЛКМ по конкретной части урока, срабатывает `LessonPartDetailSerializer`, который достает из базы весь массив лекционного текста и вопросов.

Сериализаторы готовы преобразовывать данные в JSON. Как мы поступим на следующем шаге?

- Напишем **Django Views (Представления)** на базе `APIView` или `ViewSet`, чтобы связать эти сериализаторы с URL-адресами?
- Реализуем безопасный `POST`-эндпоинт для **проверки ответов на тесты**, который будет сверять выбранный `option_id` с базой данных и возвращать результат `Правильно/Неправильно`?
- Добавим в `LessonPartDetailSerializer` поле, которое будет показывать фронтенду, **пройдена ли уже эта часть** текущим авторизованным пользователем?
- Разберем, как оптимизировать SQL-запросы в Views через `prefetch_related`, чтобы сервер не «падал» от нагрузки при сборке тяжелого JSON?



↪ `APIView`

"Давай на этой базе."

Отличный выбор. `APIView` дает полный контроль над логикой запроса, чтением методов и возвращаемыми статусами. Напишем два эндпоинта: первый будет отдавать весь контент части урока (тексты и тесты), а второй — безопасно принимать и проверять ответы на тесты от пользователя.

Создайте файл `views.py` и добавьте туда следующий код:

python

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticatedOrReadOnly,
IsAuthenticated
from django.shortcuts import get_object_or_404

from .models import LessonPart, Test, TestOption, UserTestAnswer
from .serializers import LessonPartDetailSerializer

# 1. ПОЛУЧЕНИЕ КОНТЕНТА ЧАСТИ УРОКА
class LessonPartDetailView(APIView):
    """
    Возвращает тексты и тесты для конкретной части урока.
    Доступно всем для чтения, но только авторизованным (или админам)
    для изменений.
    """
    permission_classes = [IsAuthenticatedOrReadOnly]

    def get(self, request, pk):
        # Получаем часть урока по её ID (pk) или отдаем 404, если такой
нет
        # Используем prefetch_related для оптимизации SQL-запросов
(избегаем N+1)
        lesson_part = get_object_or_404(
            LessonPart.objects.prefetch_related('texts',
'tests__options'),
            pk=pk
        )

        # Передаем объект в сериализатор
        serializer = LessonPartDetailSerializer(lesson_part)

        # Возвращаем готовый JSON на фронтенд
        return Response(serializer.data, status=status.HTTP_200_OK)

# 2. ПРОВЕРКА ОТВЕТА НА ТЕСТ
class SubmitTestAnswerView(APIView):
    """
    Принимает ответ пользователя на тест, проверяет его на сервере
и сохраняет результат в историю. Доступно только авторизованным.
    """
```

```

"""
permission_classes = [IsAuthenticated]

def post(self, request):
    # Ожидаем от фронтенда JSON вида: {"test_id": 5, "option_id":
12}

    test_id = request.data.get('test_id')
    option_id = request.data.get('option_id')

    if not test_id or not option_id:
        return Response(
            {"error": "Необходимо передать test_id и option_id."},
            status=status.HTTP_400_BAD_REQUEST
        )

    # Проверяем, существуют ли такие тест и вариант ответа в базе
    test = get_object_or_404(Test, pk=test_id)
    option = get_object_or_404(TestOption, pk=option_id,
option=test)

    # Безопасная проверка: правильный ли ответ (поле is_correct
скрыто в сериализаторе)
    is_correct = option.is_correct

    # Сохраняем попытку пользователя или обновляем старую, если он
отвечает повторно
    user_answer, created = UserTestAnswer.objects.update_or_create(
        user=request.user,
        test=test,
        defaults={
            'chosen_option': option,
            'is_correct': is_correct
        }
    )

    # Отдаем фронтенду вердикт
    return Response({
        "is_correct": is_correct,
        "explanation": test.explanation if hasattr(test,
'explanation') else "Ответ записан.",
        "message": "Правильно!" if is_correct else "Неверно,
попробуйте еще раз."
    }, status=status.HTTP_200_OK)

```



Разбор важных моментов в коде:

1. Оптимизация базы данных (`prefetch_related`)

В первом `GET`-запросе мы написали `prefetch_related('texts', 'tests__options')`. Без этого Django сделал бы один запрос за частью урока, затем по запросу на каждый текст, затем запрос на тесты и еще по запросу на варианты к каждому тесту (это классическая проблема N+1). Теперь Django сделает всего **3 быстрых запроса** и сам соберет данные в памяти.

2. Защищенная бизнес-логика

Во втором `POST`-запросе вся проверка происходит строго на бэкенде. Фронтенд шлет только ID кликнутой кнопки. Сервер сверяет `option.is_correct`, записывает результат в базу под текущего `request.user` и возвращает результат. Пользователь никак не сможет подделать ответ.

Как теперь подключить это к URL-адресам?

В файле `urls.py` вашего приложения пропишите пути к этим классам (не забудьте вызвать `.as_view()`):

python

```
from django.urls import path
from .views import LessonPartDetailView, SubmitTestAnswerView

urlpatterns = [
    # Например: /api/parts/5/ - выдаст тексты и тесты для части №5
    path('parts/<int:pk>/', LessonPartDetailView.as_view(), name='part-
detail'),

    # Например: /api/parts/submit-test/ - для отправки ответов
    path('parts/submit-test/', SubmitTestAnswerView.as_view(),
name='submit-test'),
]
```

Используйте код с осторожностью.



Логика бэкенда для вашей страницы урока практически полностью готова. В каком направлении будем двигаться дальше?

- Настроим **Django Admin**, добавив туда `TabularInline`, чтобы тексты, тесты и варианты ответов можно было удобно добавлять на одном экране?
- Добавим в вывод контента части урока флаг `is_completed`, чтобы фронтенд сразу знал, **прошел ли этот юзер** текущую часть урока?
- Напишем `APIView` для **вывода списка всех уроков** (`LessonListSerializer`), чтобы связать главную страницу сайта с этой базой?
- Разберем, как написать **автоматические тесты (APITestCase)** в Django, чтобы проверить, правильно ли работают наши новые View-классы?



"Я имел ввиду через `@api_view(['GET'])`"

Понял вас! Давайте перепишем эту логику на **функции-представления** с использованием декоратора `@api_view`.

Такой подход в Django REST Framework часто выглядит компактнее и проще читается, так как вся логика метода находится на виду без использования классов.

Вот как будет выглядеть ваш файл `views.py`:

python

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticatedOrReadOnly,
IsAuthenticated
from django.shortcuts import get_object_or_404

from .models import LessonPart, Test, TestOption, UserTestAnswer
from .serializers import LessonPartDetailSerializer
```

```

# 1. ПОЛУЧЕНИЕ КОНТЕНТА ЧАСТИ УРОКА (Функция)
@api_view(['GET'])
@permission_classes([IsAuthenticatedOrReadOnly])
def lesson_part_detail_view(request, pk):
    """
    Возвращает тексты и тесты для конкретной части урока.
    Сортировка внутри списков происходит автоматически благодаря Meta
    моделей.
    """
    # Оптимизируем запросы через prefetch_related, чтобы избежать
    # проблемы N+1
    lesson_part = get_object_or_404(
        LessonPart.objects.prefetch_related('texts', 'tests__options'),
        pk=pk
    )

    # Передаем объект в сериализатор и отдаем JSON
    serializer = LessonPartDetailSerializer(lesson_part)
    return Response(serializer.data, status=status.HTTP_200_OK)

# 2. ПРОВЕРКА ОТВЕТА НА ТЕСТ (Функция)
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def submit_test_answer_view(request):
    """
    Принимает ответ пользователя, сверяет is_correct на сервере
    и сохраняет/обновляет результат в истории.
    """
    test_id = request.data.get('test_id')
    option_id = request.data.get('option_id')

    # Валидация входных данных
    if not test_id or not option_id:
        return Response(
            {"error": "Необходимо передать test_id и option_id."},
            status=status.HTTP_400_BAD_REQUEST
        )

    # Проверяем существование теста и варианта в БД
    test = get_object_or_404(Test, pk=test_id)
    option = get_object_or_404(TestOption, pk=option_id, option=test)

    # Проверка правильности на бэкенде (безопасно для фронтенда)
    is_correct = option.is_correct

```

```

# Записываем попытку (создаем новую или перезаписываем старую)
user_answer, created = UserTestAnswer.objects.update_or_create(
    user=request.user,
    test=test,
    defaults={
        'chosen_option': option,
        'is_correct': is_correct
    }
)

# Формируем вердикт для фронтенда
return Response({
    "is_correct": is_correct,
    "message": "Правильно!" if is_correct else "Неверно, попробуйте еще раз."
}, status=status.HTTP_200_OK)

```

Используйте код с осторожностью.



Изменения в файле `urls.py`

При использовании функций декоратор `@api_view` сам превращает их в нужные Django-представления. Поэтому в файле маршрутов больше **не нужно вызывать метод** `.as_view()`:

python

```

from django.urls import path
from .views import lesson_part_detail_view, submit_test_answer_view

urlpatterns = [
    # Получение части урока по ID
    path('parts/<int:pk>/', lesson_part_detail_view, name='part-detail'),

    # Отправка ответа на тест
    path('parts/submit-test/', submit_test_answer_view, name='submit-test'),
]

```

Используйте код с осторожностью.



В чем разница для вас в коде?

- Декоратор `@api_view(['GET'])` автоматически оборачивает функцию, берет на себя обработку ошибок формата данных и позволяет возвращать объект `Response` вместо стандартного Django `JsonResponse`.
- Списки внутри возвращаемого JSON (тексты, тесты, варианты) благодаря этой функции и настроенным сериализаторам прилетят на фронтенд ровно в том порядке, который задан вашим полем `order`.

Теперь логика на функциях полностью готова к работе. Какую задачу мы разберем следующей, чтобы развить ваш проект?

- Разберем, как сделать удобное управление этим порядком (`order`) в **админ-панели Django**, чтобы не вводить цифры руками?
- Напишем функцию для **главной страницы** (`@api_view(['GET'])`), которая будет отдавать список всех доступных уроков?
- Добавим в сериализатор логику, которая проверит таблицу `UserTestAnswer` и подставит флаг, **отвечал ли уже этот юзер** на данный тест ранее?
- Посмотрим, как протестировать эти эндпоинты через утилиту **Postman** или встроенный интерфейс Django REST Framework (Browsable API)?



"Покажи как ты написал urls"

Вот как выглядит ваш файл `urls.py` для подключения функций-представлений (FBV).

В отличие от классов, при использовании функций с декоратором `@api_view` метод `.as_view()` вызывать **не нужно**.

Вариант 1. Внутри вашего Django-приложения (например, `lessons/urls.py`)

Создайте или отредактируйте файл `urls.py` прямо внутри папки вашего приложения:

python

```
from django.urls import path
# Импортируем наши функции из файла views.py
from .views import lesson_part_detail_view, submit_test_answer_view

urlpatterns = [
    # Маршрут для получения текстов и тестов конкретной части урока
    # Пример URL: http://127.0.0.0
    path('parts/<int:pk>/', lesson_part_detail_view, name='part-
detail'),

    # Маршрут для отправки ответа на тест пользователем
    # Пример URL: http://127.0.0.0
    path('parts/submit-test/', submit_test_answer_view, name='submit-
test'),
]
```

Используйте код с осторожностью.



Вариант 2. Главный файл маршрутов проекта (корневой `urls.py`)

Чтобы Django увидел маршруты вашего приложения, их нужно подключить в главный файл `urls.py` (который находится в папке рядом с `settings.py`) с помощью функции `include` :

python

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),

    # Подключаем урлы нашего приложения и задаем им общий префикс
    'api/'
    path('api/', include('имя_вашего_приложения.urls')),
]
```

Как это работает вместе:

1. Когда фронтенд делает `GET`-запрос на адрес `/api/parts/1/`, Django заходит в главный файл, видит префикс `api/`, перенаправляет запрос в `urls.py` вашего приложения, находит совпадение по `<int:pk>` (где `pk` станет равен `1`) и запускает функцию `lesson_part_detail_view`.
2. Когда пользователь отправляет `POST`-запрос на адрес `/api/parts/submit-test/`, срабатывает функция `submit_test_answer_view`.

Маршрутизация полностью настроена и готова связывать фронтенд с вашей базой данных. Какой шаг сделаем следующим для развития вашего проекта?

- Посмотрим, как протестировать эти URL-адреса через удобный встроенный браузерный интерфейс **DRF (Browsable API)**?
- Настроим **админку Django**, чтобы добавлять тексты, тесты и варианты ответов на одной странице через `TabularInline`?
- Напишем аналогичную функцию для **вывода списка всех уроков** на главной странице сайта?
- Разберем, как отправлять `POST`-запрос с фронтенда (например, через `fetch` или `axios`) на наш URL **проверки тестов**?



"Можешь помочь с ошибками?"

```
path('users/', views.user_list, name='user-list'),...
```



Эта ошибка `AttributeError` означает, что Django пытается найти функцию или класс с именем `user_list` в вашем файле `views.py`, но **не может её там обнаружить**.

Вот 3 самые частые причины, почему это происходит, и способы их решения:

1. Функция `user_list` забыта или называется иначе

Проверьте ваш файл `views.py`. Скорее всего, вы просто еще не написали эту функцию или случайно допустили опечатку в её названии (например, назвали её `users_list` во множественном числе).

Решение: Добавьте эту функцию в свой `views.py` (используя наш стиль с `@api_view`):

python

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

# Не забудьте импортировать модель User и соответствующий сериализатор

@api_view(['GET'])
def user_list(request):
    """
    Возвращает список пользователей (если это необходимо для вашего
    сайта)
    """
    # Ваша логика, например:
    # users = User.objects.all()
    # serializer = UserSerializer(users, many=True)
    # return Response(serializer.data)
    return Response({"message": "Список пользователей пока в
разработке"}, status=status.HTTP_200_OK)
```

Используйте код с осторожностью.



2. Ошибка в импорте файла `views`

Если функция `user_list` у вас точно написана в файле `views.py`, то Django может импортировать **не тот файл `views`**. Это часто случается, если структура папок проекта запутана.

В начале вашего файла `urls.py` посмотрите, как импортируются представления.

- Если написано `from . import views`, убедитесь, что этот `urls.py` находится в той же папке, что и нужный `views.py`.

- Если написано `from lessons_and_tests import views`, проверьте, совпадает ли имя приложения.

3. Вы забыли сохранить файл `views.py`

Звучит банально, но это регулярная проблема при разработке: код функции во `views.py` уже написан, но файл в редакторе (VS Code, PyCharm) еще не сохранен на диск. Нажмите `Ctrl + S` (или `Cmd + S` на Mac) во `views.py` и перезапустите сервер.

Обычно для страницы уроков список всех пользователей на бэкенде не требуется. Что мы сделаем дальше?

- Напишем представление для **списка всех уроков**, если вы случайно перепутали названия `user_list` и `lesson_list`?
- Разберем, как реализовать **регистрацию и авторизацию** пользователей, чтобы работал эндпоинт `users/`?
- Помогу исправить другие ошибки, если они возникли при запуске сервера?



"это то что я когда то тестил. Уже удалил. Давай вернемся к админ панели. Стоит ли мне с ней работать если у меня ... 

Да, работать с админ-панелью Django однозначно стоит, даже если вы создаете проект исключительно как API на Django REST Framework.

В мире веб-разработки Django ценят именно за её встроенную админку (Django Admin). Для DRF-проекта она выполняет роль **готовой системы управления контентом (CMS)**.

Почему админка жизненно необходима для вашего DRF-проекта?

1. Удобное наполнение базой данных (для авторов курсов)

Вам, как разработчику, или будущим преподавателям нужно будет как-то загружать на сайт тексты лекций, создавать тесты и варианты ответов. Делать это через консоль или SQL-запросы вручную — долго и неудобно. Админка дает готовый графический интерфейс для управления контентом «из коробки».

2. Проверка работы API

Чтобы протестировать, правильно ли ваш созданный `@api_view` отдает тексты и тесты, в базе данных сначала должны появиться тестовые данные. Проще и быстрее всего создать их через админку.

3. Управление пользователями

В админке вы сможете мгновенно сбрасывать пароли, блокировать пользователей, делать их администраторами (`is_staff`), а также смотреть, кто и когда зарегистрировался.

Как настроить админку под вашу структуру (Использование Inlines)

Поскольку ваши модели сильно связаны (`Часть урока -> Тексты -> Тесты -> Варианты`), создавать их по отдельности на разных страницах админки — мучение. Django позволяет выводить связанные таблицы прямо на одной странице с помощью механизма `Inline`.

Создайте или откройте файл `admin.py` в вашем приложении и добавьте этот код. Он позволит вам создавать часть урока, сразу писать к ней тексты и прикреплять тесты с вариантами на одном экране:

python

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .models import User, Lesson, LessonPart, Text, Test, TestOption,
UserTestAnswer

# Регистрируем кастомного пользователя, чтобы видеть дату его создания
@admin.register(User)
class CustomUserAdmin(UserAdmin):
    list_display = ('username', 'email', 'is_staff', 'created_at')
    # Добавляем наше поле пройденных уроков в интерфейс админки
    filter_horizontal = ('completed_lessons',)
```

```
# --- ИНЛАЙНЫ ДЛЯ СВЯЗАННЫХ ТАБЛИЦ ---
```

```
# Позволяет добавлять варианты ответов прямо внутри теста
```

```
class TestOptionInline(admin.TabularInline):  
    model = TestOption  
    extra = 3 # Количество пустых строк для новых ответов по умолчанию  
    ordering = ['order']
```

```
# Позволяет добавлять тексты прямо внутри части урока
```

```
class TextInline(admin.StackedInline): # StackedLayout удобнее для  
больших полей TextField  
    model = Text  
    extra = 1  
    ordering = ['order']
```

```
# Позволяет добавлять тесты прямо внутри части урока
```

```
class TestInline(admin.TabularInline):  
    model = Test  
    extra = 1  
    ordering = ['order']
```

```
# --- РЕГИСТРАЦИЯ ОСНОВНЫХ МОДЕЛЕЙ ---
```

```
@admin.register(Lesson)
```

```
class LessonAdmin(admin.ModelAdmin):  
    list_display = ('title', 'user', 'is_published', 'created_at')  
    list_filter = ('is_published', 'created_at')  
    search_fields = ('title',)
```

```
@admin.register(LessonPart)
```

```
class LessonPartAdmin(admin.ModelAdmin):  
    list_display = ('title', 'lesson', 'order')  
    list_filter = ('lesson',)  
    ordering = ['lesson', 'order']  
    # Включаем тексты и тесты прямо на страницу редактирования Части  
урока!  
    inlines = [TextInline, TestInline]
```

```
@admin.register(Test)
```

```
class TestAdmin(admin.ModelAdmin):  
    list_display = ('question', 'lesson_part', 'order')  
    # Включаем варианты ответов на страницу редактирования самого Теста  
    inlines = [TestOptionInline]
```

```
# Регистрируем историю ответов только для просмотра (read-only)
@admin.register(UserTestAnswer)
class UserTestAnswerAdmin(admin.ModelAdmin):
    list_display = ('user', 'test', 'is_correct', 'created_at')
    readonly_fields = ('user', 'test', 'chosen_option', 'is_correct',
                       'created_at')
```

Используйте код с осторожностью.



Что вы получите в итоге?

1. Зайдя на страницу любой **Части урока** в админке, вы увидите блоки для ввода текста и блоки для создания вопросов. Вам не придется переключаться между вкладками.
2. Вся рутина по управлению базой данных будет закрыта стандартными средствами Django, а вы сможете полностью сфокусироваться на написании чистого API для вашего фронтенда.

Админка настроена и готова к наполнению контентом. С какого этапа мы продолжим развивать проект?

- Разберем, как создать **Суперпользователя (Superuser)** через терминал, чтобы впервые войти в эту админку?
- Напишем `@api_view(['GET'])` функцию для **вывода списка всех уроков**, чтобы наполнить главную страницу сайта?
- Посмотрим, как сделать так, чтобы поле `order` в админке автоматически увеличивалось на +1 при создании новых записей?
- Узнаем, как подключить библиотеку `django-suit` или `django-jet`, чтобы сделать админку визуально **красивой и современной**?



"Создал суперпользователя. Я прям смогу как создавать объекты так и удалять?"

Да, абсолютно! Суперпользователь (Superuser) в Django — это «бог» вашей системы. У вас будут полные, ничем не ограниченные права на любые действия с данными.

Вы сможете зайти по адресу `http://127.0.0.1`, ввести логин с паролем и делать следующее:

Что вы сможете делать в админке:

- **Создавать (Create)**: Добавлять новые уроки, части уроков, писать тексты лекций и заводить тестовые вопросы с вариантами ответов.
- **Редактировать (Update)**: Менять тексты, переназначать порядок через поле `order`, исправлять опечатки в вопросах.
- **Удалять (Delete)**: Удалять ошибочно созданные объекты.

Важное предупреждение про каскадное удаление (`on_delete=models.CASCADE`)

Поскольку в ваших моделях настроена жесткая связь `on_delete=models.CASCADE`, будьте аккуратны при удалении крупных объектов:

- Если вы удалите **Урок**, Django автоматически удалит **все его части**, а вместе с ними — все связанные **тексты, тесты и варианты ответов**.
- Если вы удалите одну **Часть урока**, автоматически удалятся только её тексты и тесты, но сам урок останется целым.

Перед каждым удалением админка Django услужливо покажет вам предупреждающий экран со списком всех объектов, которые будут стерты заодно, так что вы случайно ничего не потеряете.

Поздравляю, теперь у вас есть полностью рабочая панель управления контентом! Наполните базу парой тестовых уроков, чтобы нам было с чем работать.

Какую задачу вы хотите решить следующей?

- Напишем функцию `@api_view(['GET'])` для **вывода списка всех уроков**, чтобы оживить главную страницу вашего сайта?
- Разберем, как прямо из фронтенда (с помощью JavaScript/React/Vue) делать **авторизацию** под вашим суперпользователем, чтобы получать доступ к защищенным эндпоинтам?

- Добавим в сериализаторы логику, которая будет автоматически подставлять **имя автора** (текущего юзера) при создании уроков?
- Посмотрим, как прямо в браузере протестировать работу нашего эндпоинта `lesson_part_detail_view` через встроенный интерфейс **DRF (Browsable API)**?



→ Напишем функцию `@api_view(['GET'])` для вывода списка всех уроков, чтобы оживить главную страницу вашего сайта?

"Напиши пожалуйста это"

Вот код для вывода списка всех уроков на главной странице. Мы напишем функцию-представление с декоратором `@api_view(['GET'])` и применим оптимизацию, чтобы база данных работала максимально быстро.

1. Добавление во `views.py`

Добавьте эту функцию в ваш файл `views.py`. Мы будем использовать `LessonListSerializer`, который мы спроектировали ранее (он выводит название, автора и количество частей):

python

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import AllowAny # Доступно всем без
авторизации
from django.db.models import Count

from .models import Lesson
from .serializers import LessonListSerializer


@api_view(['GET'])
@permission_classes([AllowAny]) # Страницу со списком уроков могут
видеть даже неавторизованные гости
def lesson_list_view(request):
    """
```

```

Возвращает список всех опубликованных уроков для главной страницы.
Уроки отсортированы: сначала новые (по дате создания).
"""

# Оптимизация:
# 1. select_related('user') сразу подгружает авторов (избегаем N+1
для юзеров)
# 2. annotate(parts_count=Count('parts')) считает количество частей
прямо на уровне базы данных SQL

lessons = Lesson.objects.filter(is_published=True)\
    .select_related('user')\
    .annotate(parts_count=Count('parts'))\
    .order_by('-created_at')

# Передаем список в сериализатор. Флаг many=True обязателен для
коллекций!
serializer = LessonListSerializer(lessons, many=True)

return Response(serializer.data, status=status.HTTP_200_OK)

```

Используйте код с осторожностью.



⚠ Важная корректировка в serializers.py

Чтобы оптимизация из `views.py` сработала идеально, нам нужно слегка скорректировать поле `parts_count` в `LessonListSerializer` (внутри файла `serializers.py`).

Вместо медленного `source='parts.count'`, который заставляет Django делать отдельный SQL-запрос для каждого урока, мы укажем чтение аннотированного поля, которое мы посчитали в View:

python

```

class LessonListSerializer(serializers.ModelSerializer):
    user = serializers.ReadOnlyField(source='user.username')
    # Указываем read_only=True, значение автоматически подтянется из
    .annotate(parts_count=...)
    parts_count = serializers.IntegerField(read_only=True)

    class Meta:
        model = Lesson

```

```
fields = ['id', 'title', 'user', 'parts_count',  
'is_published', 'created_at']
```

Используйте код с осторожностью.



Подключение к `urls.py`

Теперь регистрируем этот эндпоинт в файле `urls.py` вашего приложения:

python

```
from django.urls import path  
from .views import lesson_part_detail_view, submit_test_answer_view,  
lesson_list_view  
  
urlpatterns = [  
    # Главная страница со списком всех уроков  
    # Пример URL: http://127.0.0.0  
    path('lessons/', lesson_list_view, name='lesson-list'),  
  
    # Ваши предыдущие урлы  
    path('parts/<int:pk>', lesson_part_detail_view, name='part-  
detail'),  
    path('parts/submit-test/', submit_test_answer_view, name='submit-  
test'),  
]
```

Используйте код с осторожностью.



Как это работает и что увидит фронтенд?

Когда ваш фронтенд (например, на React/Vue или через простой `fetch`) сделает `GET`-запрос на `/api/lessons/`, он получит чистый и быстрый JSON-ответ:

json

```
[
  {
    "id": 2,
    "title": "Введение в Django REST Framework",
    "user": "admin",
    "parts_count": 5,
    "is_published": true,
    "created_at": "2026-06-23T12:00:00Z"
  },
  {
    "id": 1,
    "title": "Основы языка Python",
    "user": "teacher_alex",
    "parts_count": 3,
    "is_published": true,
    "created_at": "2026-06-22T09:30:00Z"
  }
]
```

Используйте код с осторожностью.



Фронтенд сможет легко пробежаться по этому массиву циклом и отрисовать красивые карточки уроков, показывая пользователю название, автора и сколько частей внутри его ждет.

Эндпоинт для главной страницы готов. Какой шаг сделаем следующим для полноценной работы сайта?

- Напишем функцию для **детального просмотра урока**, которая покажет список всех его частей (чтобы пользователь мог кликать по ним ЛКМ)?
- Разберем, как сделать так, чтобы суперпользователь видел в списке **все уроки**, а обычные студенты — **только опубликованные** (`is_published=True`)?
- Реализуем механизм **регистрации и авторизации** (например, через Simple JWT), чтобы ваши защищенные тесты начали принимать реальных пользователей?
- Посмотрим, как прямо из браузера протестировать этот эндпоинт в красивом интерфейсе **Django REST Framework**?




```
↳ from .views import lesson_part_detail_view,  
    submit_test_answer_view, lesson_li
```

"Я импортировал следующим образом. Можешь писать в этом же ключе?from . import views"

Принято! Импорт через `from . import views` — это отличный профессиональный стиль, так как он защищает код от конфликтов имен и сразу показывает, что функции берутся из текущего приложения.

Вот обновленный и чистый пример того, как теперь будет выглядеть ваш файл `urls.py` с использованием этого стиля импорта:

python

```
from django.urls import path  
# Импортируем модуль views целиком из текущей папки  
from . import views  
  
urlpatterns = [  
    # Главная страница со списком всех уроков  
    # URL: http://127.0.0  
    path('lessons/', views.lesson_list_view, name='lesson-list'),  
  
    # Получение текстов и тестов конкретной части урока  
    # URL: http://127.0.0  
    path('parts/<int:pk>/', views.lesson_part_detail_view, name='part-  
detail'),  
  
    # Отправка ответа на тест  
    # URL: http://127.0.0  
    path('parts/submit-test/', views.submit_test_answer_view,  
name='submit-test'),  
]
```

Используйте код с осторожностью.



В таком же ключе мы будем писать и все последующие маршруты.

Блок навигации и базовые эндпоинты полностью настроены. Какую задачу мы разберем следующей, сохраняя этот стиль кода?

- Напишем функцию `views.lesson_detail_view` для **детального просмотра урока**, которая отдаст фронтенду список всех его частей для кликанья ЛКМ?
- Реализуем в файле `views.py` систему **регистрации и авторизации** пользователей (например, через токены), чтобы можно было безопасно сдавать тесты?
- Добавим во `views.lesson_part_detail_view` проверку, чтобы сервер возвращал информацию о том, **отвечал ли уже студент** на эти тесты раньше?
- Посмотрим, как настроить **CORS-политику** в `settings.py`, чтобы ваш фронтенд (React/Vue/HTML) мог без ошибок делать запросы к этим эндпоинтам?



↪ Напишем функцию `views.lesson_detail_view` для детального просмотра урока, которая отдаст фронтенду список всех его...

"C CORS разобрался заранее.`CORS_ALLOWED_ORIGINS`
`= [ 'http://localhost:3000', 'http://127.0.0.1:3000', ...`



Отлично, что вы заранее настроили `CORS_ALLOWED_ORIGINS` для портов `3000` и `3001` — это сэкономит много времени при интеграции фронтенда (например, на React или Vue).

Реализуем функцию `lesson_detail_view`. Когда пользователь кликает на карточку урока на главной странице, фронтенд запрашивает этот эндпоинт, получает список всех его частей с их `id` и `title` и выводит их в виде меню/списка. При клике ЛКМ по любой части фронтенд уже будет запрашивать детальную информацию через `lesson_part_detail_view`, который мы написали ранее.

1. Добавление во `views.py`

Добавьте эту функцию в ваш файл `views.py`. Мы используем `LessonDetailSerializer`, а также метод `prefetch_related('parts')` для оптимизации запроса к базе данных:

python

```
@api_view(['GET'])
@permission_classes([AllowAny]) # Список частей урока могут видеть все
зости сайта
def lesson_detail_view(request, pk):
    """
    Возвращает информацию о конкретном уроке и список всех его частей
    (только заголовки и ID) для отображения меню навигации.
    """
    # Получаем урок по ID и сразу подгружаем все связанные части
    (оптимизация)
    lesson = get_object_or_404(
        Lesson.objects.filter(is_published=True).prefetch_related('parts'),
        pk=pk
    )

    # Передаем объект в сериализатор
    serializer = LessonDetailSerializer(lesson)
    return Response(serializer.data, status=status.HTTP_200_OK)
```

Используйте код с осторожностью.



⚠ Важная корректировка в serializers.py

В предыдущем шаге мы набросали базовый LessonDetailSerializer, который выводил только строковые названия частей через SlugRelatedField. Для фронтенда этого мало — нам обязательно нужно передавать id каждой части, чтобы фронтенд знал, какой именно ID подставлять в URL при клике ЛКМ.

Давайте сделаем для этого маленький промежуточный сериализатор. Измените LessonDetailSerializer в вашем файле serializers.py следующим образом:

python

```
# Компактный сериализатор для вывода частей внутри урока
class LessonPartShortSerializer(serializers.ModelSerializer):
    class Meta:
        model = LessonPart
```

```

        fields = ['id', 'title', 'order'] # Передаем id для
        формирования ссылки на фронтенде

# Обновленный сериализатор деталей урока
class LessonDetailSerializer(serializers.ModelSerializer):
    user = serializers.ReadOnlyField(source='user.username')
    # Используем созданный выше сериализатор вместо SlugRelatedField
    parts = LessonPartShortSerializer(many=True, read_only=True)

    class Meta:
        model = Lesson
        fields = ['id', 'title', 'user', 'is_published', 'parts',
        'created_at']

```

Используйте код с осторожностью.



2. Добавление в `urls.py` (в вашем стиле)

Теперь подключаем новую функцию в файл `urls.py` строго через `from . import views`:

python

```

from django.urls import path
from . import views

urlpatterns = [
    # Главная страница со списком всех уроков
    path('lessons/', views.lesson_list_view, name='lesson-list'),

    # Детальная страница урока (список его частей для кликанья ЛКМ)
    # Пример URL: http://127.0.0.1
    path('lessons/<int:pk>/', views.lesson_detail_view, name='lesson-
detail'),

    # Получение текстов и тестов конкретной части урока
    path('parts/<int:pk>/', views.lesson_part_detail_view, name='part-
detail'),

    # Отправка ответа на тест
    path('parts/submit-test/', views.submit_test_answer_view,

```

```
name='submit-test'),  
]
```

Используйте код с осторожностью.



Что теперь увидит фронтенд на странице урока?

При `GET`-запросе на `/api/lessons/1/` бэкенд вернет чистый JSON:

json

```
{  
  "id": 1,  
  "title": "Основы языка Python",  
  "user": "teacher_alex",  
  "is_published": true,  
  "created_at": "2026-06-22T09:30:00Z",  
  "parts": [  
    {  
      "id": 12,  
      "title": "Установка и первая программа",  
      "order": 1  
    },  
    {  
      "id": 13,  
      "title": "Переменные и типы данных",  
      "order": 2  
    },  
    {  
      "id": 14,  
      "title": "Условные операторы if-else",  
      "order": 3  
    }  
  ]  
}
```

Используйте код с осторожностью.



Фронтенд берет массив `parts`, строит из него боковое меню, а при клике ЛКМ, например, по второй части, делает запрос на `/api/parts/13/` и отрисовывает по центру прямоугольник с текстом и кнопкой переключения на тесты. Архитектурная цепочка полностью замкнулась!

Вся базовая логика отображения контента готова. Какую задачу мы разберем следующей?

- Настроим систему **регистрации и авторизации** (например, через JWT токены), чтобы бэкенд понимал, какой именно юзер сейчас проходит тесты?
- Добавим в вывод контента части урока логику, которая проверит, **выполнял ли уже студент эти тесты** ранее, чтобы показать его старые ответы?
- Посмотрим, как сделать так, чтобы преподаватели (авторы уроков) могли видеть в списке **даже свои неопубликованные уроки** (`is_published=False`), а студенты — только опубликованные?
- Разберем, как написать простой скрипт на фронтенде (на чистом JS / `fetch`), чтобы **протестировать отправку ответов** на бэкенд?



"Можешь проверить `serializers.py`?"

Давайте проведем полный аудит вашего файла `serializers.py`. На основе всех изменений, которые мы обсудили, я собрал финальный, чистый и оптимизированный код.

Ниже представлен проверенный вариант без ошибок импорта или структуры.

Проверенный и готовый `serializers.py`

python

```
from rest_framework import serializers
from .models import Lesson, LessonPart, Text, Test, TestOption
```

1. СЕРИАЛИЗАТОР ВАРИАНТОВ ОТВЕТОВ

```
class TestOptionSerializer(serializers.ModelSerializer):
    class Meta:
```

```
        model = TestOption
```

```
        # Передаем id (нужен фронту для отправки POST-запроса) и текст
```

варианта.

Поле is_correct ИСКЛЮЧЕНО из соображений безопасности (чтобы не подсмотрели в коде).

```
fields = ['id', 'answer', 'order']
```

2. СЕРИАЛИЗАТОР ТЕСТОВ (Включает в себя варианты ответов)

```
class TestSerializer(serializers.ModelSerializer):
    # Подтягиваем связанные варианты ответов через
    related_name='options'
    options = TestOptionSerializer(many=True, read_only=True)

    class Meta:
        model = Test
        fields = ['id', 'question', 'order', 'options']
```

3. СЕРИАЛИЗАТОР ТЕКСТОВ

```
class TextSerializer(serializers.ModelSerializer):
    class Meta:
        model = Text
        fields = ['id', 'lesson_material', 'order']
```

4. СЕРИАЛИЗАТОР ДЕТАЛЕЙ ЧАСТИ УРОКА (Тексты + Тесты для центрального прямоугольника)

```
class LessonPartDetailSerializer(serializers.ModelSerializer):
    # Объединяем тексты и тесты внутри одной части урока через их
    related_name
    texts = TextSerializer(many=True, read_only=True)
    tests = TestSerializer(many=True, read_only=True)

    class Meta:
        model = LessonPart
        fields = ['id', 'title', 'order', 'texts', 'tests']
```

5. СЕРИАЛИЗАТОР ЧАСТЕЙ УРОКА ДЛЯ НАВИГАЦИИ (Компактный)

```
class LessonPartShortSerializer(serializers.ModelSerializer):
    class Meta:
        model = LessonPart
        fields = ['id', 'title', 'order']
```

6. СЕРИАЛИЗАТОР ДЛЯ КАРТОЧКИ УРОКА (На главной странице)

```
class LessonListSerializer(serializers.ModelSerializer):
    # Выводим строковое имя автора вместо числового ID
```

```

user = serializers.ReadOnlyField(source='user.username')
# Поле подтянет значение из .annotate(parts_count=Count('parts'))
во views.py
parts_count = serializers.IntegerField(read_only=True)

class Meta:
    model = Lesson
    fields = ['id', 'title', 'user', 'parts_count', 'is_published',
'created_at']

# 7. СЕРИАЛИЗАТОР ДЕТАЛЕЙ УРОКА (Выдает метаданные урока + список его
частей для меню)
class LessonDetailSerializer(serializers.ModelSerializer):
    user = serializers.ReadOnlyField(source='user.username')
    # Используем созданный выше компактный сериализатор частей
    parts = LessonPartShortSerializer(many=True, read_only=True)

    class Meta:
        model = Lesson
        fields = ['id', 'title', 'user', 'is_published', 'parts',
'created_at']

```

Используйте код с осторожностью.



Результаты проверки (Чек-лист безопасности и производительности):

1. Безопасность тестов (Пройдено успешно)

В `TestOptionSerializer` отсутствует поле `is_correct`. Студенты не смогут схитрить, открыв вкладку *Network* в браузере.

2. Иерархия связей (Пройдено успешно)

Все вложенные сериализаторы (`tests` , `texts` , `options` , `parts`) используют правильные имена, совпадающие с параметрами `related_name` в ваших моделях.

3. Автоматическая сортировка фронтенда (Пройдено успешно)

Так как вы прописали `ordering = ['order', 'id']` внутри `class Meta` ваших моделей, DRF автоматически отсортирует массивы `texts` , `tests` и `options` перед отправкой. Фронтенду останется просто вывести их по порядку.

4. Оптимизация производительности (Пройдено успешно)

Разделение на `LessonPartDetailSerializer` (тяжелый контент) и `LessonPartShortSerializer` (только `id` и `title` для меню) защитит сервер от избыточных запросов при обычной навигации по сайту.

Файлы `models.py`, `serializers.py`, `views.py` и `urls.py` теперь полностью синхронизированы и готовы к запуску. Какую задачу мы разберем следующей?

- Настроим **регистрацию и авторизацию** (например, через популярную библиотеку Simple JWT), чтобы заработал эндпоинт отправки ответов?
- Добавим в `TestSerializer` информацию о том, **отвечал ли уже текущий пользователь на этот тест** и какой вариант он выбирал?
- Посмотрим, как прямо на фронтенде написать **функцию fetch/axios запроса**, которая заберет этот JSON и передаст его в ваше меню?
- Разберем, как написать **автоматические тесты** на Python для проверки этих сериализаторов?



Настроим регистрацию и авторизацию (например, через популярную библиотеку Simple JWT), чтобы заработал...

"Расскажи вот это"



Для настройки регистрации и авторизации в Django REST Framework мы будем использовать **Simple JWT**. Это современный стандарт для SPA-приложений (на React, Vue и т.д.). Он выдает пользователю два токена: `access` (короткоживущий, для каждого запроса) и `refresh` (для обновления первого).

Ниже пошаговое руководство, написанное в вашем фирменном стиле (с импортом `from . import views`).

Шаг 1. Установка библиотеки

Установите пакет через терминал:

```
bash
```

```
pip install djangorestframework-simplejwt
```

Используйте код с осторожностью.



Шаг 2. Настройка `settings.py`

Добавьте JWT в настройки вашего проекта:

```
python
```

```
from datetime import timedelta
```

```
# 1. Подключаем аутентификацию DRF
```

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    )  
}
```

```
# 2. Настраиваем время жизни токенов
```

```
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),      # Время жизни  
access-токена  
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),        # Время жизни  
refresh-токена  
    'ROTATE_REFRESH_TOKENS': False,  
    'BLACKLIST_AFTER_ROTATION': False,  
    'AUTH_HEADER_TYPES': ('Bearer',),                  # Тип заголовка  
в запросах: Bearer <токен>  
}
```

Используйте код с осторожностью.



Шаг 3. Создание сериализатора регистрации в

`serializers.py`

Добавьте в конец вашего файла `serializers.py` класс для регистрации новых пользователей:

python

```
from .models import User

class RegisterSerializer(serializers.ModelSerializer):
    # Поле пароля делаем только для записи, чтобы оно не возвращалось в JSON
    password = serializers.CharField(write_only=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password']

    def create(self, validated_data):
        # Метод create_user автоматически хеширует пароль в базе данных
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data.get('email', ''),
            password=validated_data['password']
        )
        return user
```

Используйте код с осторожностью.



Шаг 4. Написание функций во `views.py`

Теперь напишем функцию регистрации. Функции для входа (логина) нам писать не нужно — Simple JWT предоставляет их в готовом виде!

Добавьте в ваш `views.py`:

python

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
```

```

from rest_framework.permissions import AllowAny

from .serializers import RegisterSerializer

@api_view(['POST'])
@permission_classes([AllowAny]) # Регистрация доступна всем гостям
def register_view(request):
    """
    Регистрация нового пользователя в системе.
    """
    serializer = RegisterSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(
            {"message": "Пользователь успешно зарегистрирован!"},
            status=status.HTTP_201_CREATED
        )
    return Response(serializer.errors,
                    status=status.HTTP_400_BAD_REQUEST)

```

Используйте код с осторожностью.



Шаг 5. Подключение маршрутов в `urls.py`

Подключим эндпоинты в вашем стиле `from . import views`. Готовые представления Simple JWT импортируем отдельно:

```

python

from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView
from . import views # Ваш привычный стиль импорта

urlpatterns = [
    # 1. Авторизация (Вход) – возвращает access и refresh токены
    # URL: http://127.0.0.0
    path('token/', TokenObtainPairView.as_view(),
    name='token_obtain_pair'),

    # 2. Обновление access-токена с помощью refresh-токена
    # URL: http://127.0.0.0refresh/
    path('token/refresh/', TokenRefreshView.as_view(),

```

```
name='token_refresh'),

# 3. Регистрация нового пользователя
# URL: http://127.0.0
path('register/', views.register_view, name='auth_register'),

# Ваши старые URL-адреса уроков
path('lessons/', views.lesson_list_view, name='lesson-list'),
path('lessons/<int:pk>', views.lesson_detail_view, name='lesson-
detail'),
path('parts/<int:pk>', views.lesson_part_detail_view, name='part-
detail'),
path('parts/submit-test/', views.submit_test_answer_view,
name='submit-test'),
]
```

Используйте код с осторожностью.



Как это тестировать на фронтенде / в Postman:

1. **Регистрация:** Отправляете POST на `/api/register/` с JSON `{"username": "alex", "password": "superpassword"}`.
2. **Вход:** Отправляете POST на `/api/token/` с теми же данными. Бэкенд вернет два токена: `"access"` и `"refresh"`.
3. **Защищенные запросы (Сдача тестов):** Чтобы отправить ответ на тест через функцию `submit_test_answer_view`, ваш фронтенд должен добавить в заголовки (Headers) запроса полученный access-токен:

text

Authorization: Bearer <ваш_access_токен>

Используйте код с осторожностью.



Система пользователей и авторизации полностью интегрирована в проект!
Как мы поступим теперь?

- Посмотрим, как на стороне фронтенда **сохранять эти токены** (в `localStorage`) и автоматически прикреплять к запросам?

- Добавим в сериализаторы тестов вывод информации о том, **какие тесты этот юзер уже решал ранее** (на основе его токена)?
- Разберем, как сделать так, чтобы при прохождении всех частей урока, этот урок **автоматически добавлялся в** `completed_lessons` пользователя?
- Настроим **валидацию сложности пароля** при регистрации, чтобы пользователи не вводили слишком простые пароли?

